JAVA 项目开发报告

1. 项目概述

基本内容

本项目为一款基于 Java 开发的 2D 冒险游戏,回忆童年经典:《魂斗罗》，结合了用户登录系统、排行榜、BOSS 战及 GUI 界面设计。游戏目标是提供完整的用户体验，包括玩家登录、游戏玩法及结束界面等功能。

主要参加人员

- 组长： 23331156 颜佳熙
- 成员：

  23331165 余飞

  23331153 许汝欢

  23331181 周毅

业绩权重说明

本项目的业绩权重分配方案：1 2 2 2。

工作任务的分解和人员分工

- **颜佳熙：** 负责游戏主逻辑代码的编写，包括角色移动、攻击、碰撞检测及游戏流程控制。
- **余飞：** 负责 BOSS 战的开发，增加多阶段的 BOSS 攻击模式及难度设计，编写胜利界面。
- **许汝欢：** 负责云服务器上 MySQL 数据库的部署，开发用户登录界面、账号密码验证、分数和时间统计，以及天梯排行榜界面。
- **周毅：** 负责 GUI 界面设计，包括游戏的视觉效果和用户交互。当前尚未完成开始菜单和游戏结束菜单的功能优化。

2. 业务需求分析和基本模块功能

业务需求分析

- 实现用户账号系统：支持注册、登录功能。
- 提供流畅的游戏体验：包括主游戏流程、BOSS 战等内容。
- 添加用户统计功能：记录游戏分数、时间，展示排行榜。
- 优化 GUI 界面，提高用户体验。

基本模块功能

1. **用户管理模块**

用户注册、登录验证。分数和时间统计。

2. **主逻辑模块**

角色移动、攻击功能。碰撞检测及角色状态切换。

3. **BOSS 战模块**

多阶段攻击逻辑。战斗结束后显示胜利界面。

4. **数据库模块**

使用 MySQL 存储用户数据、分数统计。

5. **排行榜模块**

天梯排行榜功能，展示用户排名。

6. **GUI 界面模块**

开始菜单、操作提示、游戏结束菜单及整体界面设计。

3. 系统设计与实现

计算机体系设计（采用技术和平台）

- **开发语言：** Java
- **数据库：** MySQL
- **开发工具：** IntelliJ IDEA, pxcook 定位画面位置,git 团队协同合作.
- **技术：** JavaFX,awt,swing,css 界面美化
- **云平台：** 使用云服务器部署 MySQL 数据库

功能详细设计与数据库字典

(1)角色移动功能

功能描述

负责通过键盘输入处理角色的移动，更新角色的坐标并触发相关的动画状态。

输入

用户键盘输入 (KeyEvent)，包括 leftDown、rightDown、upDown、downDown 的按下或释放状态。

输出

更新角色在游戏场景中的位置（Hero.position）。更新角色当前状态（例如行走、跳跃、站立等）。

实现步骤

键盘事件监听:通过 KeyListener 捕获键盘事件，将方向键状态保存到 leftDown、rightDown 等布尔变量中。**状态判断:**根据按下的键组合判断角色的移动方向（左、右、上、下），并设置相应速度或跳跃的逻辑。**位置更新:** 使用 Hero.move() 方法按帧计算新位置，确保角色不会越界。**状态动画触发:** 根据角色位置更新其显示状态，例如行走、跳跃等动画。

(2)攻击功能

功能描述

通过用户输入触发角色射击，生成子弹并加入 heroBullets 列表，同时检查武器类型和子弹数量限制。

输入

用户按下射击键（KeyEvent，默认 J）。当前武器类型（Hero.weapon）。

输出

新生成的子弹加入 heroBullets 列表。更新角色的射击状态（Hero.canShot）。

实现步骤

键盘事件监听:捕获 J 键按下事件并检查 Hero.canShot 是否为 true。**子弹生成:**调用 Hero.shot() 方法，根据当前武器类型生成子弹对象。**子弹数量限制:**根据武器类型限制子弹的生成数量（如普通武器最多 6 发）。**加入子弹队列:**将生成的子弹对象添加到 heroBullets 列表中。**冷却设置:**将 Hero.canShot 设置为 false，防止过频射击。

(3)碰撞检测功能

功能描述

检测角色与敌人、子弹、场景元素的碰撞，触发对应的游戏逻辑（如扣血、销毁等）。

输入

当前角色位置（Hero.position）。

子弹列表（heroBullets 和 enemyBullets）。

敌人列表（enemys）。

输出

更新角色或敌人的状态（如血量减少、销毁等）。

从子弹或敌人列表中移除已命中的对象。

实现步骤

获取游戏对象位置:遍历 heroBullets 和 enemyBullets 列表,获取每个子弹的当前位置。**碰撞逻辑判断:**使用矩形区域的重叠检测判断是否发生碰撞。**碰撞结果处理:**如果子弹命中敌人，从 enemys 列表中移除该敌人并增加分数。如果子弹命中角色，减少角色生命值。子弹命中后从相应列表中移除。

(4)游戏流程控制

功能描述

根据游戏状态（如角色死亡、Boss 出现）控制场景切换、音效播放和胜利/失败逻辑。

输入

游戏状态标志位（如 bossBattle、showBossScene）。游戏计时器（timeCont）。

输出

更新游戏场景（scenePosition）。触发音效或显示 UI 界面（如胜利界面）。

实现步骤

场景切换:根据 mapPosition 和 scenePosition 判断是否需要进入新场景或 Boss 战场景。**状态更新:**根据 heroNum 判断角色是否存活，触发游戏结束或重生逻辑。**音效播放:**在 bossBattle 状态下触发 Boss 战背景音乐。**胜利条件检查:**如果 boss.isDefeated() 返回 true，切换到胜利界面并播放胜利音效。

(5)敌人初始化功能

功能描述

在游戏初始化时生成多个敌人，设置敌人的初始位置、状态和类型。

输入

敌人初始位置和类型列表（如 enemy0、enemy1 等）。

输出

添加到 enemys 列表的敌人对象。

实现步骤

创建敌人对象:遍历敌人初始数据（如位置、类型），实例化 SimpleEnemy 对象。**敌人列表:**将创建的敌人对象逐一加入 enemys 列表中。

(6)JavaFX 的界面布局

布局方式
- 使用了 AnchorPane, VBox, 和 GridPane 等 JavaFX 的布局容器，适配不同的内容布局需求。
- AnchorPane 用于固定位置的组件（如菜单按钮和标题）。
- GridPane 用于网格布局显示排行榜数据，分行列整齐排列。

- VBox 用于垂直排列按钮和其他内容。

样式与美化
- 使用了 CSS 样式表（通过 scene.getStylesheets().add 方法加载）来统一管理样式，例如字体、颜色、间距等。

(7)动画效果

按钮悬停动画
- 利用 ScaleTransition 实现按钮的缩放动画，增强用户交互体验。
- 动画通过设置 OnMouseEntered 和 OnMouseExited 事件触发。

标题动画
- 菜单标题在加载时添加了 ScaleTransition，使其具有缩放的进入效果，吸引用户注意。

(8)排行榜功能

数据库交互
- 使用了一个 Manager 类，通过 getTop5UsersByScore 方法从数据库中获取前 5 名玩家数据。
- 数据库的异常处理通过 try-catch 结构管理。

动态生成排行榜内容
- 通过遍历 List<User>，动态创建 GridPane 行，显示用户名、分数和时间。
- 排行榜每一行的数据项通过列约束（ColumnConstraints）和行约束（RowConstraints）进行宽高调整。

(9)事件绑定

各个按钮绑定了对应的事件处理器：
- startGameButton 调用 startGame 方法启动游戏。
- exitButton 调用 System.exit(0) 退出程序。
- rankButton 显示排行榜界面（使用了 showRankingUI 方法）。
- backButton 返回菜单界面。

(10)多场景管理

通过 Scene 和 Stage 的切换管理多个界面：
- 菜单界面（showMenu 方法）。
- 排行榜界面（showRankingUI 方法）。

使用 Stage 的 setScene 方法动态切换场景。

(11)游戏启动逻辑

多线程
- 启动游戏的逻辑使用了一个新的线程 Thread game，运行游戏主框架 MainFrame。
- 使用 game.join() 等待线程结束，确保游戏生命周期与主界面独立。

游戏结束回调
- 游戏结束后，通过 App.gameEnd 方法记录游戏结果（用户名、分数和计时器格式化后的时间）。

(12)分隔符实现

利用 Line 对象，创建了垂直的分隔线，用于视觉上分割排行榜的列。
createVerticalSeparator 方法封装了分隔线的创建逻辑，提升代码可重用性。

(13)对象抽象与类层次图

App.java：主要负责登录、注册及用户界面切换逻辑。

MainFrame.java：实现游戏主逻辑，包括角色移动、攻击及 BOSS 战等功能。

抽象类有三个：Bullet.java,Player.java,Weapon.java

Simple,Strong,Shot,NunlinerBullet 和 Weapon 继承 Bullet.java 和 Weapon.java,SimpleEnemy 和 Hero 继承 Player.java

编码实现

示例代码：awt 绘图的双缓冲机制

```
public void update(Graphics g)
{
   //将 Graphics 对象转换为 Graphics2D，以便使用更多绘图功能。
   Graphics2D g2 = (Graphics2D)g;
   //如果是第一次调用，初始化画布资源：
   if (this.firstTime) {
```

```
        Dimension dim = getSize();
        int w = dim.width;
        int h = dim.height;
        new Rectangle(dim);
        this.bi = ((BufferedImage)createImage(w, h));
        this.big = this.bi.createGraphics();
        this.firstTime = false;
        this.big.setStroke(new BasicStroke(1.0F));
    }
    //绘制背景图（Imgs.BACKGROUND01），动态调整背景位置和显示的大小。
    //目标区域的左上角 (dx1, dy1)决定图像绘制在屏幕的起始位置。目标区域的右下角 (dx2, dy2)。决定图像在屏幕上的宽度和高度。后面四个是源图像的左上和右下,672 其实是带着一点放大的背景,原图片也就 224
    //这个时候 this.big 还没绘制完成
    this.big.drawImage(Imgs.BACKGROUND01, (int)this.scenePosition.x, (int)this.scenePosition.y, this.backgroundSizeOfWidth, (int)this.scenePosition.y + 672, (int)this.mapPosition, 0, (int)this.mapPosition + this.backgroundSizeOfWidth / 3, 224, null);
    drawNPC(this.big);
    if (this.showBossScene) {
        playPreBossBattleAnimation(this.big);
    }
    if (this.bossBattle) {
        playBossBattleAnimation(this.big);
    }
    for (int i = 0; i < this.enemys.size(); i++) {
        ((SimpleEnemy)this.enemys.get(i)).drawPlayer(this.big, this.mapPosition);
    }
    if (this.bossChildren.size() > 0) {
        playBossChildrenAnimation(this.big);
    }

    if (!this.hero.visible) {
        //设置是否可见(可被攻击)
        this.big.setComposite(this.alphaComposite);
    }
    this.hero.drawPlayer(this.big, this.mapPosition);
    this.big.setComposite(this.alphaComposite2);
    drawHeroBullets(this.big);
    drawEnemyBullets(this.big);
    this.big.setColor(Color.WHITE);
    this.big.setFont(new Font(null, 1, 28));
    g2.setColor(Color.GRAY);
    g2.fillRect(0, 0, getWidth(), getHeight());
    //this.big 是 Graphics2D，用于操作后缓冲。
```

//g2 是 Graphics2D，用于操作屏幕画布。
g2.drawImage(this.bi,(int)this.bgPosition.x,(int)this.bgPosition.y,(int)(this.backgroundSizeOfWidth / 1.3D), (int)(this.backgroundSizeOfHeight / 1.3D), this);
}

双缓冲的“双”：前缓冲：显示在屏幕上的画面。后缓冲：内存中的画布，用于准备下一帧内容。所有绘图操作都在后缓冲中完成。 绘制完成后，将后缓冲的内容一次性复制到前缓冲（屏幕）。避免了直接在屏幕上多次绘图带来的闪烁现象,BufferedImage 是内存中的一块画布，用于存储最终的图像。它是绘图内容的容器。
以我的代码为例子：this.big：操作后缓冲的绘图工具，是从 this.bi.createGraphics() 得到的 Graphics2D 对象。draw 参数一直是 this.big,this.big 是从 BufferedImage 创建的 Graphics2D 对象,避免直接在屏幕上绘制，减少屏幕闪烁。通过 BufferedImage，可以保存整个画面状态，方便后续操作（如缩放、重绘等）它负责在后缓冲（this.bi）上绘制所有内容。绘制顺序严格按照代码书写的顺序：背景 → NPC → 主角 → 子弹等。后绘制的内容会覆盖先绘制的内容。全部都绘制在 this.big 之后，this.bi 将先在内存画布中完成的所有绘制操作，最后统一渲染到屏幕。

4. 系统测试

测试目标

- 验证登录和注册功能的正确性。
- 测试主游戏逻辑：角色移动、攻击、BOSS 战。
- 检查数据库分数记录与排行榜的展示功能。
- 界面测试：操作提示和菜单的优化需求。

测试结果

1. **用户登录模块：** 功能正常，用户名和密码验证无误。
2. **主游戏逻辑：** 角色移动、碰撞检测及攻击功能正常，但存在视口宽度不自适应屏幕的缺陷。
3. **BOSS 战模块：** 攻击逻辑正常。
4. **界面优化：**

缺陷：放大屏幕时，视口宽度不自适应屏幕。

5. 项目展望

未来的项目优化目标包括：

1. **功能完善：**

完善胜利界面，提升玩家游戏体验。

2. **界面优化：**

调整操作提示的位置，使其更加美观。实现视口宽度自适应屏幕功能。

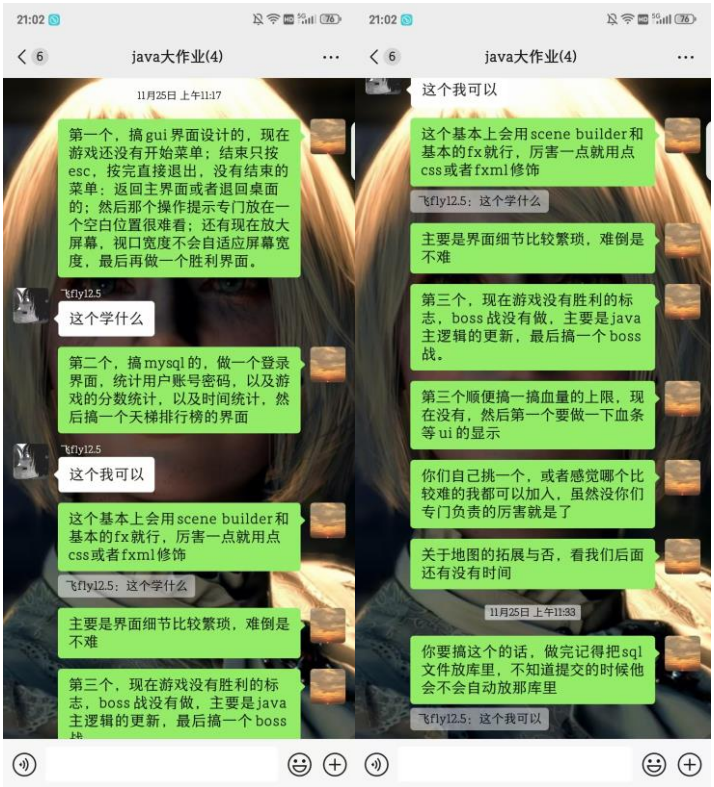
3. **性能提升：**

优化游戏运行性能，减少卡顿现象。

6. 附录：组会内容纪要

由于我们基本平时使用 github 进行代码的整合和推送,有问题会在微信群内及时交流,所以并没有开过多的组会.

第一次组会

- **时间：** 11 月 25 日
- **内容：** 确定项目分工，讨论项目需求及技术选型。
- **结果：** 明确各成员的任务，确定开发工具及数据库方案。
- 

第二次组会

- **时间：** 12 月 10 日
- **内容：** 进度汇报及问题讨论,整合项目开发报告。
- **结果：**完成用户登录模块和主游戏框架。提出界面优化和功能完善需求。